

基于 Verilog 语言的数字钟实现

北京师范大学 杨博文

(北京师范大学人工智能学院 北京市海淀区 100875)

摘要：本文基于 Verilog 语言实现了一个数字钟的设计，支持五大功能：计时、重置、整点报时、闹钟设定、时钟暂停。本文详细分析了输入输出需求及功能实现逻辑，并通过模块化变成和逻辑优化提高了代码的可读性和效率。设计过程包括了输入输出格式分析、逻辑实现、测试验证，最终通过仿真展示了数字钟的工作原理。

关键词：数字钟；数字逻辑；Verilog。

第一章 功能分析

本题目中要求我们实现数字钟电路的设计，应当包括计时、重置、整点报时、设定闹钟、时钟暂停共计 5 个功能。根据我们所需要的功能和输出格式，我们对输入、输出、实现作出以下分析。

一、输出格式与计时功能

1. 输入与输出格式分析说明

由于 Verilog 语言并不具有直接输出十进制数字的功能，我们对于每一位数字使用两位二进制来表示。即，对于“时时-分分-秒秒”的格式而言，分别对应 hourHigh、hourLow、minuteHigh、minuteLow、secondHigh、secondLow 六个二位二进制数字。

我们希望一天有 $3 \times 3 = 9$ 小时，1 小时有 9 分钟，1 分钟有 9 秒。故对于上述六个二进制数字中的每一个，都有 00、01、10 三种状态。每次时钟应当向前的时候，根据时钟逻辑进行变化。

在这一部分，我们需要的输入有 1 个，即 cp，代表向前跳钟的信号。由于 vwf 的限制，我们以 1ns 为周期设定该信号。

与其他功能密切相关的输入输出我们将在对应章节进行说明。

2. 计时功能的分析实现

在计时功能的实现中，我们实际上就是考虑每次 cp 产生上边沿的时候，让秒针跳 1 位。根据计时规则，当下一位从 10 变化回 00 的时候，我们要将上一位向前进 1。即：当秒从 0111 往后再走的时候，秒要变为 1000。

在实现上，我们选择 case 进行选择，对每一种情况进行分类并且确定接下来的赋值。Case 选择完后，我们将使用 if 来判断是否应当进位，也就是如果本来是 10，下一次要变回 00，此时我们对上一位进行 case 判断，依次类推。

事实上，这一部分可以使用元件例化的方法，定义向前走模块，将 case 的过程进行浓缩。但是多写多错，所以我选择简单的进行复制粘贴更改。

二、重置功能

1. 输入输出分析

作者简介：杨博文（2004-），男，北京师范大学本科在读，主修计算机科学与技术，辅修数学与应用数学。

在重置过程中，我们使用输入值 `reset`。如果 `reset` 为 1，将所有时钟清 0。

2. 功能实现

由于同一个变量不能在两个 `always` 中被赋值，所以我们需要在同一个 `always` 中对 `reset` 进行判断。在所有的对于输出变量的操作中，我们选择使用非阻塞赋值。当 `reset` 为 1 时，对所有的变量赋值为 `2'b00`。

三、整点报时与设定闹钟功能

由于这两个都属于报时功能，故这里一起写。实际上就是当时钟与设定的时间相同的时候，输出一个信号。

1. 输入输出分析

对于整点报时功能，我们定义输出 `baoshi`。对于设定闹钟功能，我们输入 6 个 2 位二进制数字，分别对应我们时钟的 6 个 2 位二进制数，如果这 6 个数全部一样的话，我们就输出闹钟，这里的输出为 `ifAlarm`。

6 个 2 位二进制数按照“时时-分分-秒秒”的顺序，分别称为 `hourHighAlarm`、`hourLowAlarm`、`minuteHighAlarm`、`minuteLowAlarm`、`secondHighAlarm`、`secondLowAlarm`。

2. 功能实现

我们将这两个功能做成了两个不同的逻辑。

对于整点报时功能而言，我们将输出夹在了时钟向前走这一步中。事实上就是，如果涉及到了“时”的变化，那必然是到达了一个新的整点，此时设定 `baoshi` 为 1，否则为 0。

对于设定闹钟的功能而言，我们建立了一个新的 `always`，当六个输入或者 `secondLow` 也就是秒的低位变化的时候，进行判断。如果六个数字对应相等，则输出 `ifAlarm` 为 1，反之则为 0。

四、时钟暂停功能

尽管这个功能看上去简单，但我愿意称其为最难的功能。

1. 输入输出分析

我们将输入 `ifpause` 作为是否暂停的标志。如果 `ifpause` 为 1，计时功能暂停，时钟维持不变。由于我们的时钟会随着 `cp` 上升沿而变化，所以考虑能不能通过改变时钟信号进而暂停计时功能。我们想到：如果将 `ifpause==1` 时的时钟设置为恒 0，那就解决了问题。可是直接这样真的可行吗？

2. 功能实现

我们设定中间变量 `reg stillclk` 作为我们的新的时钟。当 `cp`、`ifpause` 变化的时候（利用 `always`），`stillclk` 进行变化。当 `ifpause==1` 时，`stillclk=0`（不变化）；反之，`stillclk=cp`。

但是，检查的时候发现了问题，即：在暂停期间重置不会重置。这是因为我们的重置是在时钟上升沿的 `always` 里面判断的，当暂停的时候不会产生上升沿。所以，在产生 `stillclk` 的 `always` 中，我们加入对于 `reset` 的检测，并且 `stillclk=0` 不仅要求 `ifpause==1` 还要求 `reset==0`。这样的话我们就解决了检测中出现的问题。

第二章 代码实例

```
module bowenyang    //这里是主模块，所有的输入输出
(
    input wire cp,
    input wire reset,
    input wire ifpause,
    input wire[1:0] hourHighAlarm,
    input wire[1:0] hourLowAlarm,
    input wire[1:0] minuteHighAlarm,
    input wire[1:0] minuteLowAlarm,
    input wire[1:0] secondHighAlarm,
    input wire[1:0] secondLowAlarm,
    output reg baoshi,
    output reg ifAlarm,
    output reg[1:0] hourHigh,
    output reg[1:0] hourLow,
    output reg[1:0] minuteHigh,
    output reg[1:0] minuteLow,
    output reg[1:0] secondHigh,
    output reg[1:0] secondLow
);

reg stillclk;    //中间变量
always @(cp,reset,ifpause) begin    //新时钟的构建，即暂停的实现
    if(ifpause==1'b1 & reset==1'b0) begin
        stillclk=0;
    end
    else begin
        stillclk=cp;
    end
end

always @(posedge stillclk) begin    // 计时、重置、整点报时的实现
    baoshi=0;
    case(secondLow)
        2'b00:secondLow<=2'b01;
        2'b01:secondLow<=2'b10;
        2'b10:secondLow<=2'b00;
        default:secondLow<=2'b00;
    endcase
    if(secondLow==2'b10) begin
```

```

case(secondHigh)
    2'b00:secondHigh<=2'b01;
    2'b01:secondHigh<=2'b10;
    2'b10:secondHigh<=2'b00;
    default:secondHigh<=2'b00;
endcase
if(secondHigh==2'b10) begin
    case(minuteLow)
        2'b00:minuteLow<=2'b01;
        2'b01:minuteLow<=2'b10;
        2'b10:minuteLow<=2'b00;
        default:minuteLow<=2'b00;
    endcase
    if(minuteLow==2'b10) begin
        case(minuteHigh)
            2'b00:minuteHigh<=2'b01;
            2'b01:minuteHigh<=2'b10;
            2'b10:minuteHigh<=2'b00;
            default:minuteHigh<=2'b00;
        endcase
        if(minuteHigh==2'b10) begin
            baoshi=1;
            case(hourLow)
                2'b00:hourLow<=2'b01;
                2'b01:hourLow<=2'b10;
                2'b10:hourLow<=2'b00;
                default:hourLow<=2'b00;
            endcase
            if(hourLow==2'b10) begin
                case(hourHigh)
                    2'b00:hourHigh<=2'b01;
                    2'b01:hourHigh<=2'b10;
                    2'b10:hourHigh<=2'b00;
                    default:hourHigh<=2'b00;
                endcase
            end
        end
    end
end
end
if(reset==1'b1) begin
    secondLow<=2'b00;
    secondHigh<=2'b00;

```

```

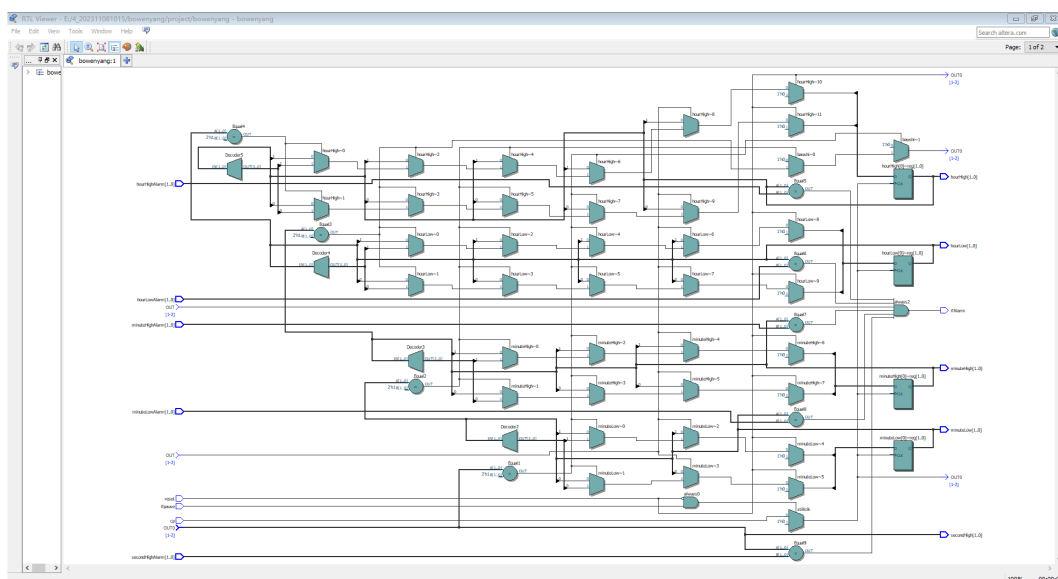
        minuteLow<=2'b00;
        minuteHigh<=2'b00;
        hourLow<=2'b00;
        hourHigh<=2'b00;
    end
end

always
@((hourHighAlarm,hourLowAlarm,minuteHighAlarm,minuteLowAlarm,secondHighAlarm,secondLowAlarm,secondLow) begin // 闹钟的实现
    if(hourHighAlarm==hourHigh & hourLowAlarm==hourLow & minuteHighAlarm==minuteHigh &
        minuteLowAlarm==minuteLow & secondHighAlarm==secondHigh & secondLowAlarm==secondLow)
    begin
        ifAlarm=1'b1;
    end
    else begin
        ifAlarm=1'b0;
    end
end
end

endmodule

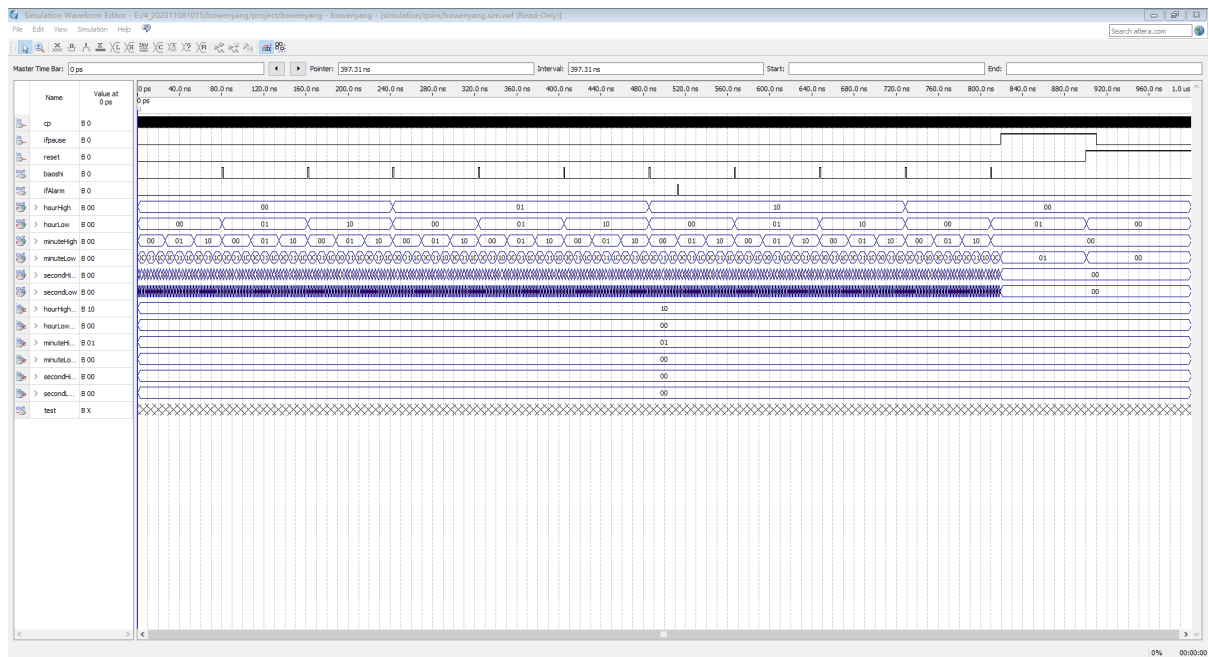
```

第三章 设计框图



第四章 输出介绍

Vwf 文件运行如下：



这之中，输入 cp 以 1ns 为周期，闹钟设定为 20:01:00。

我确实不知道该写啥了……就这样吧。